LecScribe: Draft Project Idea

Nov 7, 2026 · @Aaron Mathew

The problem

Students can't listen and take good notes at the same time, and most AI note tools send lecture audio to someone else's servers.

- **What gets lost:** slides hold the headings, but the explanations, examples and hints are spoken, and they vanish.
- **Why recordings don't fix it:** an hour of audio is rarely rewatched.
- **Why local matters:** cloud transcription means lecturer and student audio leaves the room, which raises privacy and consent questions, and it needs a connection that lecture halls may not reliably have.
- **Fit with the brief:** this is an everyday problem for the University of Manchester community, solved with AI that runs 100% on a device we control.

The idea

LecScribe is a lecture companion that listens on your own laptop, follows the slides, and writes margin notes, with no cloud and no accounts.

- **Follows along:** it shows the slides and moves to the next one when the lecturer does.
- **Margin notes:** background agents turn what the lecturer *said* about each slide into small sticky notes tied to that slide.
- **Keeps what matters:** the student clicks a sticky to save it, and saved notes are browsable by module and week.
- **Shares peer to peer:** coursemates join the live session directly, with no server in between.
- **Closes the loop:** students tap "I'm lost" on a slide, and the lecturer can click Explain slide to record a fresh explanation that is summarised and pinned to that slide.
- **Catches you up:** if you miss a lecture, a coursemate can share a link to that week's notes.

Assumption: one laptop in the lecture hall is the host and runs the models; other devices only receive slides and notes. Open question: is the host the presenter's laptop or a student's?

How it works

A lecture session runs in seven steps; steps 1 to 5 happen on the host laptop and peers join at step 7.

1. **Load slides.** The host uploads the PDF. The app extracts each slide's text (QVAC OCR for slides that are only images) and keeps page images for display.
2. **Embed temporarily.** Slide text is embedded with QVAC and held in memory for the session only.
3. **Listen.** Microphone audio goes through QVAC speech-to-text in short chunks.
4. **Track the slide.** Each transcript chunk is compared with the slide embeddings. The app advances only when the match is confident, and the host can always override.
5. **Summarise asynchronously.** One background agent per slide collects what was said while that slide is up, compares it with the slide text, and writes sticky notes for points the slide does not contain.
6. **Save on click.** Clicking a sticky saves it to the student's notebook under module and week.
7. **Share.** The notes feed and slide images are published to a Pears room that coursemates join by key or QR code.

Lecturer feedback: "I'm lost" and Explain slide

When students are lost on a slide, the lecturer can click Explain slide, and the app turns the fresh explanation into a note pinned to that slide.

1. **Students signal.** A student taps "I'm lost" on the slide they are viewing. Each device writes to its own Pears log, and the host merges them.
2. **Lecturer sees the heat.** A heatmap strip over the slide thumbnails shows tap counts per slide, darker meaning more. The lecturer sees counts, not names.
3. **Lecturer clicks Explain slide.** The app starts a dedicated recording for that slide, separate from the normal listening stream.
4. **Analyse locally.** QVAC speech-to-text transcribes the explanation, and the local LLM summarises it in plain language, using the slide text as context.
5. **Pin it.** The summary appears beside the slide as a distinct "Lecturer explanation" note, is sent to every connected device, and students can click to save it like any other sticky.

The recording stops when the lecturer clicks Done or moves to another slide.

Why it matters: the fix comes from the lecturer's own words at the moment of confusion, not a generic AI explanation.

Assumption: the lecturer runs the host laptop. If a student hosts instead, the lecturer needs a separate lecturer view on their own device.

Demo note: with few real users, simulate the taps from a second device or a script so the heatmap lights up on cue.

Student library: modules, weeks and sharing

Each student's LecScribe opens as a directory of modules and weeks, every week has its own vector store, and a missed week can be filled in from a coursemate's share link.

The directory has three levels:

- **Module**
  - **Week**, marked attended, missed or imported
    - Slides as shown by the host
    - Saved sticky notes and lecturer explanations
    - Lecture summary and transcript
    - That week's own vector store

Week state	How it shows	What the student can do
Attended, notes saved	Note count and a summary badge	Open, search, share
Attended, nothing saved	Empty week	Open the slides
Missed	Greyed out	Paste a share link to import a coursemate's notes
Imported	Tagged "shared by a coursemate"	Open and search

Independent stores. Deleting, sharing or rebuilding one week never touches another. Search is scoped to the current week, and a search across weeks queries each store in turn. Slide embeddings used for live tracking stay temporary, as in How it works; the week's store holds saved notes, summaries and transcript chunks.

Sharing a missed lecture:

1. A student who attended opens the week and clicks Share, which creates a link (a Pears key) for that week's notes.
2. They send the link to a coursemate by any channel.

3. The coursemate pastes it into the missed week, and LecScribe pulls the notes directly from the sharer's device.
4. The notes appear in that week tagged as shared, and are embedded locally into the week's own vector store.

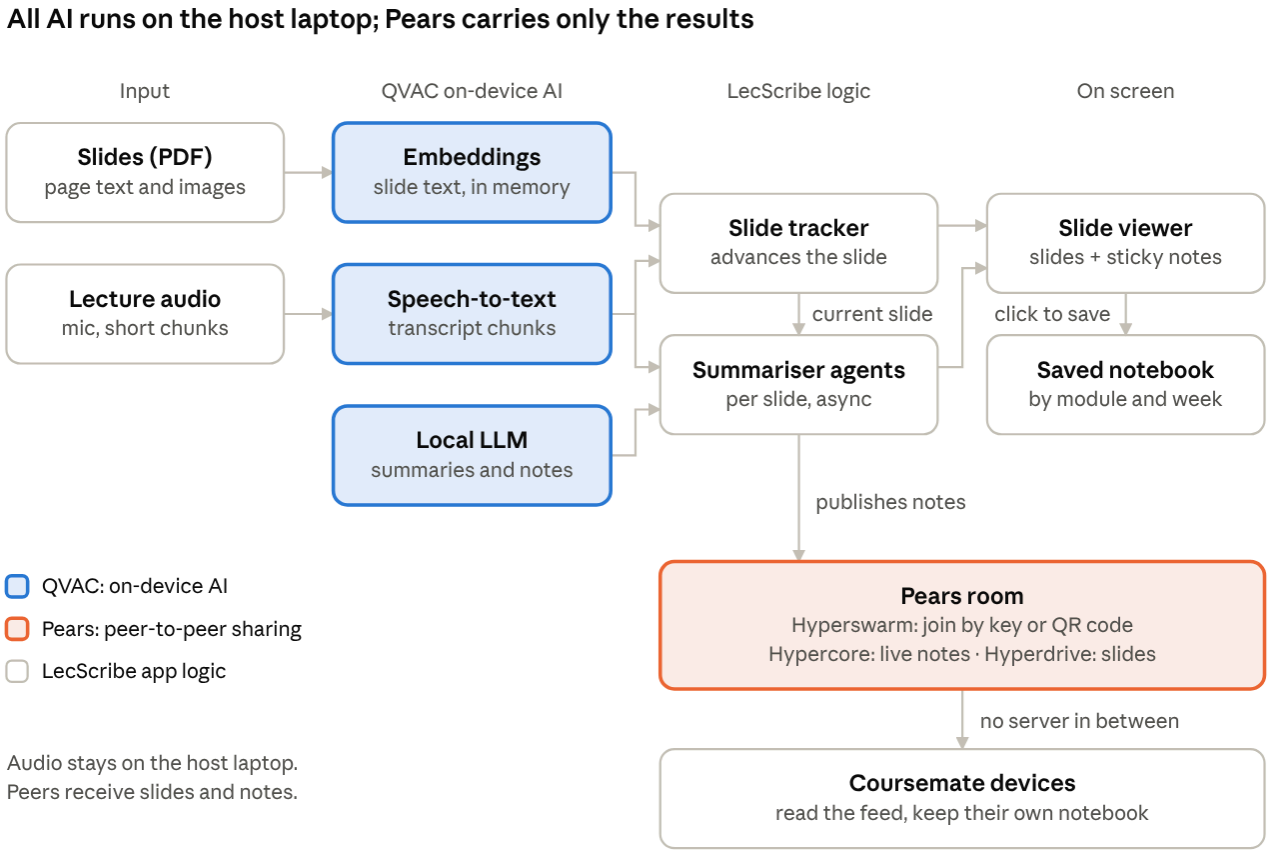
We share text, not vectors, so the receiving device builds embeddings with its own on-device model.

Limits: the link only works while a device holding the notes is online, and the sharer chooses what goes in.

Assumption: a week keeps transcripts and summaries rather than raw audio. Open question: should audio recordings be kept as well?

Architecture

The host laptop runs every model through the QVAC SDK, and Pears carries only slides and notes to other devices.



architecture · local AI on the host, P2P sharing to peers

Read it left to right: slides and audio feed QVAC, the app logic drives the screen, and Pears shares the notes.

Judging criteria map

Every Tether criterion has a concrete answer, and the two 25% criteria carry half the score.

Criterion	Weight	How LecScribe answers it
Problem & Impact	25%	A daily student problem, solved privately
QVAC implementation	25%	Speech-to-text, embeddings and a local LLM, all through the QVAC SDK with no external APIs
Pears implementation	10%	Live notes and slides shared peer to peer through a joinable room
Technical architecture	5%	Host runs the models, peers only receive data, summaries run asynchronously
Creativity & Originality	10%	Notes only for what was said but is not on the slide
Design & Experience	15%	Slides with sticky notes, click to save, browse by module and week
Presentation	10%	Repeatable demo from a recorded lecture, with the internet switched off for the AI part

Demo plan

A pre-recorded lecture clip makes the three-minute demo repeatable, because a live microphone in a noisy room is the likeliest thing to fail.

1. **0:00** State the problem in one sentence, then show the app with a slide deck loaded.
2. **0:20** Switch the host laptop's internet off, to show nothing depends on the cloud.
3. **0:40** Play the lecture clip. The app follows the slides and advances on its own.
4. **1:30** Sticky notes appear for things said but not on the slide. Click one to save it.
5. **2:00** Open the notebook and browse saved notes by module and week.
6. **2:20** A second device joins the Pears room and receives the live notes.
7. **2:45** Say plainly what works and what is still rough.

Backup: a screen recording of the full run, in case anything fails on stage.

Risks and honest limits

The biggest risks are slide tracking, local model speed and peer discovery, and each has a fallback.

Risk	Fallback
Slide advances at the wrong moment	Confidence threshold, bias towards the next slide, manual override always available
Speech-to-text errors in a noisy room	Demo with a clean recorded clip; show the transcript beside each note
Local LLM too slow, so notes lag	Small model, short prompts, agents run asynchronously and notes appear when ready
AI notes that are wrong	Label notes as AI-generated and link each one to its transcript segment
Peer discovery fails on campus Wi-Fi	Unverified whether it works with no internet; test early, use a phone hotspot as fallback
QVAC and Pears APIs are new to us	Start from the hello-pear-qvac-tui template and attend the 16:00 Theatre A workshop

Build order and stretch goals

Build the single-device version first, so a working demo exists before any peer-to-peer work starts.

1. Slide viewer with manual next and previous.
2. Local speech-to-text on a recorded clip.
3. Slide embeddings and automatic slide tracking.
4. Summariser agents and sticky notes, with click to save.
5. Module and week directory, each week with its own vector store.
6. Pears room sharing live notes and slides to a second device.
7. "I'm lost" taps, the heatmap, and Explain slide.
8. Share link for importing a missed week's notes.
9. UI polish and demo rehearsal.

Stretch goals, most valuable first:

- **Ask across weeks:** local search over saved notes, with a jump back to the slide.
- **Exam-emphasis flag:** pin anything introduced with phrases like "this will come up".

- **Flashcard export:** saved stickies become flashcards.
- **Live translation captions** for international students.